

SEATING CHART – 23 GROUPS

FRONT OF ROOM

Projector / Screen

TA's – RESERVED SEATING (First Bench)

Group 1

Group 3

Group 5

Group 7

Group 9

Group 11

Group 13

Group 15

Group 17

Group 19

Group 21

Group 23

Group 2

Group 4

Group 6

Group 8

Group 10

Group 12

Group 14

Group 16

Group 18

Group 20

Group 22

AISLE

BACK OF ROOM

Entrance / Exit

Announcements

- Teams formed. Mentors assigned
- Sit with your teammates according to the seating chart
- Sign up for Google Cloud Credits ASAP. Details in an Ed post.
- Office Hours have begun. Schedule on Canvas – in CCB 267
- GPT APIs out for each group. Sign up in class today if you haven't already
- Sign up for GitHub Pro.
- Project 1 Assignments Released

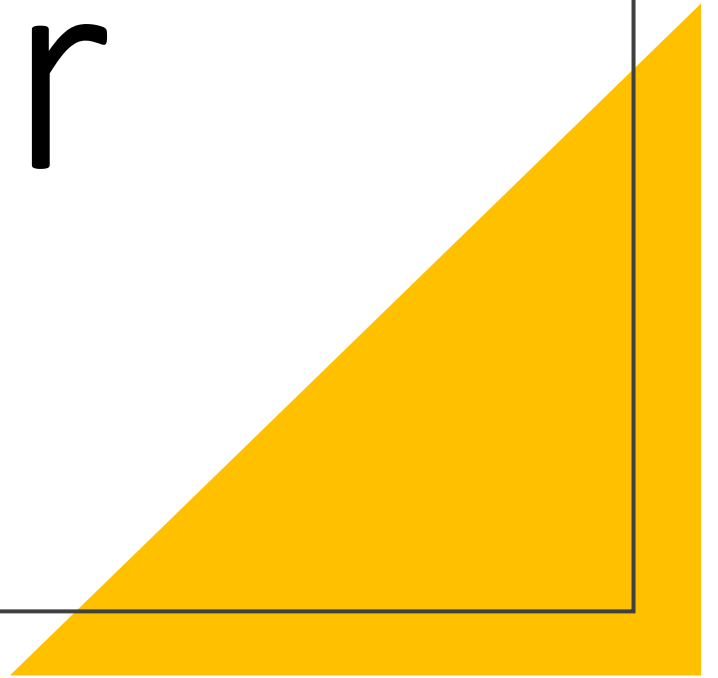
CS3300 A: Introduction to Software Engineering

Lecture 03: Tools of the Trade #1

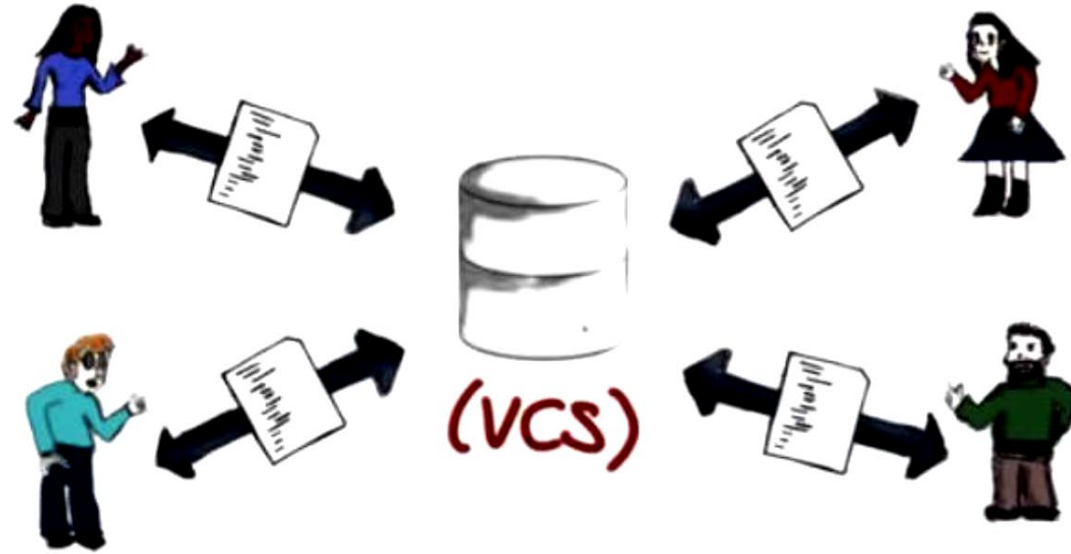
Version Control Systems, GIT, Code Review, GitHub
Actions

Dr. Nimisha Roy ▶ nroy9@gatech.edu

Git Refresher



What are Version Control Systems?



- A tool that software developers use for keeping track of revisions of their project
 - snapshots of your project over time.
 - Documents, source files etc.
- Most obvious benefits:
 - Option to go back and revisit
 - Collaborate with multiple people

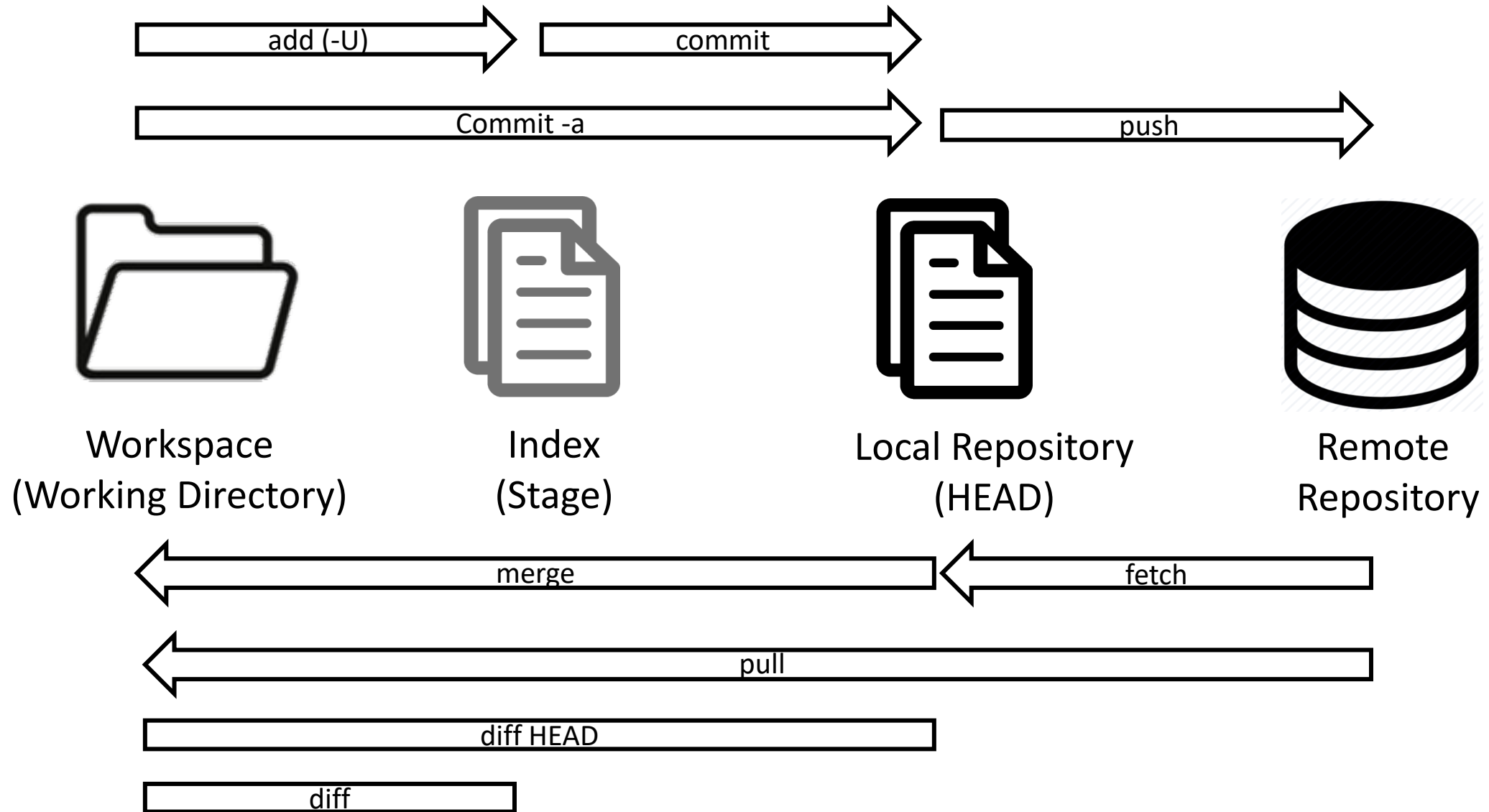
Importance

- **Enforce Discipline:** Manages process by which control of items passes from one person to another
- **Archive versions:** store subsequent versions of source-controlled items
- **Maintain Historical Information:** Author of a specific version; date and time of a specific version; etc. Retrieve and compare.
- **Enables Collaboration:** share data, files and documents
- **Recover from accidental edits/revisions**
- **Conserve Disk Space:** centralizing the management of the version.
 - Instead of having many copies spread around, one or a few central points where these copies are stored
 - efficient algorithms to store changes, so keep many versions without taking up too much space.

Don'ts in VCS

- Adding Derived Files
 - E.g., executable file derived from compiling a set of source codes
 - No reason to add it
- Adding bulky binary files
 - Try to keep them local
- Creating a local copy of files/tree of files
 - Don't do this!!
 - Useless, risky, confusing
 - Trust the version control system

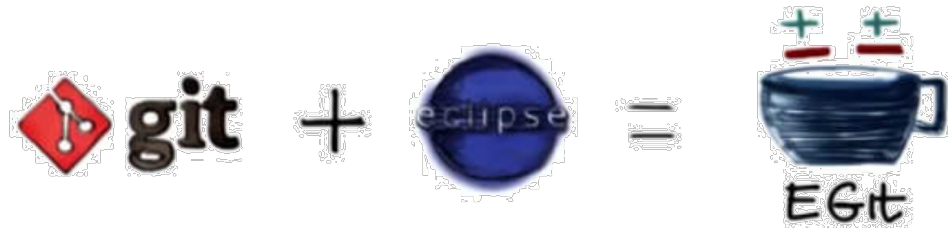
GIT Workflow Recap



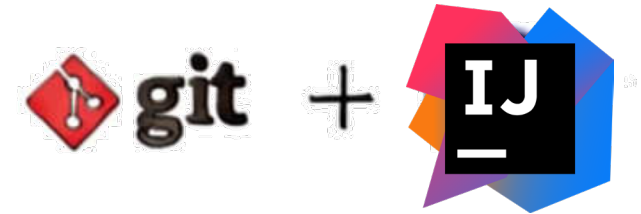
GIT Plugins

Install GIT: Follow instructions on <https://git-scm.com/book/en/v2/Getting-Started-Installing-Git>

Egit: GIT Plugin available for Eclipse; can be downloaded at www.eclipse.github.com and can be installed in Eclipse



GitToolBox for IntelliJ: The plugin can be downloaded [here](#).



Visual Studio Code has integrated source control management (SCM) and includes [Git](#) support out-of-the-box.



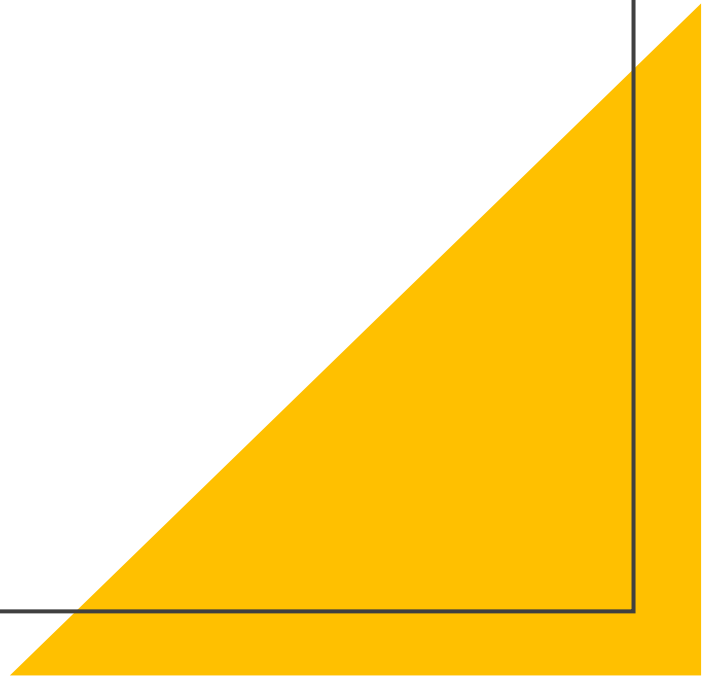
GitHub

- GIT hosting website. Get an account and create your remote repositories
- GitHub repository for your projects
- Provides easy-to-use FREE desktop clients for Mac and Windows
(<https://desktop.github.com>)
- **GitHub Pages:**
 - One click to enable for your GitHub repo.
 - Hosted directly from your GitHub repository.
 - Just edit, push, and your changes are live.
 - This course's website is a GitHub Page.
- ALWAYS SET YOUR GITHUB REPOSITORY TO BE PRIVATE, UNLESS YOU ARE ABSOLUTELY SURE YOU WANT IT PUBLIC !!!

Git Basics Demo

notes on website

Branches, Merge Conflict, Code Review



GIT Demo – Creating Branches

- By default, when you create your project you will be on main/master
- It is good practice to have different branches for different features, people, etc.
- To see all local branches:
 - `git branch`
- To create a new branch:
 - `git branch [BRANCHNAME]`
- To move to (checkout) a branch:
 - `git checkout [BRANCHNAME]`
- To create a new branch and move to it:
 - `git checkout -b [BRANCHNAME]`

GIT Demo – Merging Branches

- Merging allows you to carry the changes in one branch over to another branch, combining both branches

To merge two branches:

1. `git checkout [NAME_OF_BRANCH_TO_MERGE_INTO]`
2. `git merge [NAME_OF_BRANCH_TO_BRING_IN]`

Example: merging *feature* branch into *master* branch:

1. `git checkout master`
2. `git merge feature`

Why Code Reviews?

- Improve Overall Quality of Code
 - Having eyes on source code that you didn't write can help identify issues
- Facilitating Team Collaboration
 - Checking out each other's code better helps you understand how each feature is implemented
- Identifying bugs early in process
- Good for onboarding new developers to establish best practices within the organization
- Significant % of your time in your job is code maintenance.

Pull Requests

- Tool to aggregate branch changes and request that the changes be merged into a different branch.
- Done through the GitHub GUI

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#).

 base: master < compare: dev < ✓ Able to merge. These branches can be automatically merged.



Merges sprint 6 changes into master

Write

Preview

H B I ≡ <> 🔗 ≡ ≡ ☑ @ ↗ ↶

Sprint 6 changes include:

- ...

- ...

- ...

Attach files by dragging & dropping, selecting or pasting them. 

Create pull request <

Branch Protection

Protected branches ensure that collaborators on your repository cannot make irrevocable changes to branches. Enabling protected branches also allows you to enable other optional checks and requirements, like required status checks and required reviews.

Always have a branch protection rule enforced in your main GitHub repository branch. Settings → Branches → Branch Protection Rule → Require a pull request before merging

Note: This is only possible for public repositories (with GitHub free) and private repositories (with GitHub Pro). **So, sign up for GitHub Pro** (https://education.github.com/discount_requests/application)

Pull Requests: Good Rules of Thumb

Keep PRs small, focused, and reviewable

- One logical change per PR — one feature, bug fix, or refactor
- Aim for roughly < 300–500 lines changed when practical
- Use a few meaningful commits rather than many “fix” commits
- Write a clear PR description:
 - What changed?
 - Why was it needed?
 - How was it tested?
- Include/update tests with the code
- Request review when the change is ready for meaningful feedback
- Use Draft PRs for early feedback on work in progress
- Before merging: CI passes + required review approved

Bad PR vs. Good PR

Bad PR	Good PR
“Updated application”	“Add password reset flow”
1,800 lines touching several unrelated features	250 lines focused on one change
17 commits: fix, oops, try again, final...	3 meaningful commits
No explanation of why the change was made	Clear what / why / testing description
Tests added “later”	Tests included with the implementation
Reviewer has to figure out where to start	PR highlights important files/decisions
Review requested only after a huge feature is finished	Draft PR used for early feedback when useful
Failed CI can be ignored and merged	Branch protection requires CI + approval

Code Review Assignment: Creating Branches, Pull Requests, Performing Reviews

Both for Projects 1 and 2:

1. Create Separate Branches for every feature
 - a. You might create sub branches of these branches as you implement new portions of each feature
2. Perform a Pull Request
3. Reviewing Code and Closing Pull Requests
4. Merging Branches [Do not delete them until the assignment is graded]

GitHub Actions



GitHub Actions

- GitHub Actions is an event-driven workflow automation product for supporting the software development process in the GitHub environment.
- Using GitHub Action , we can:
 - Automate SDLC (Software Development Lifecycle) workflows
 - Implement CI/CD, DevOps



Anatomy of a GitHub Action

```
name: Example workflow
```

```
on: push
```

```
jobs:
```

```
  build:
```

```
    runs-on:
```

```
      steps:
```

```
        - uses: actions/checkout@v2
```

```
        ...
```

Workflow

Triggered by event



Job



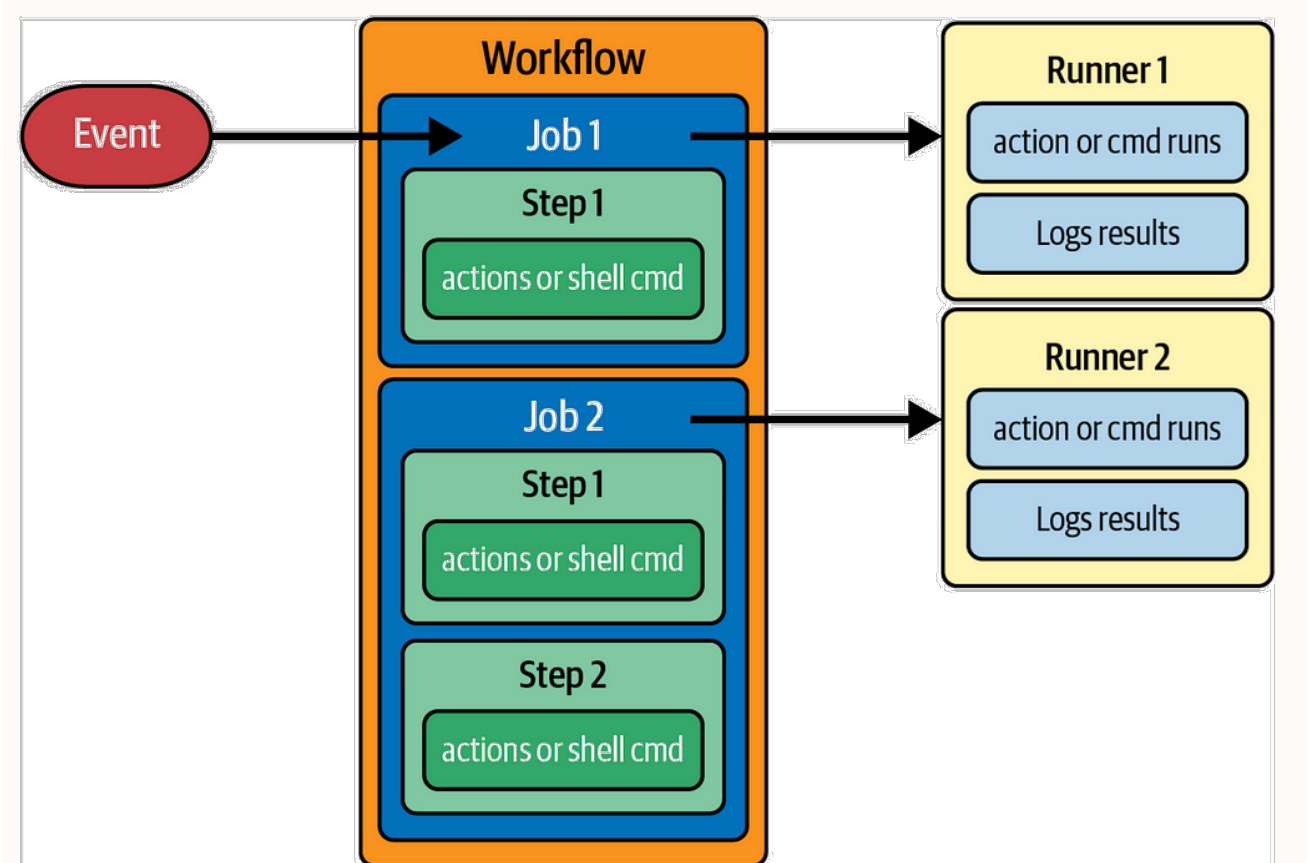
Step



Action

Anatomy of a GitHub Action

- A workflow is a unit of automation from start to finish. It contains one or more jobs.
- A workflow is triggered by an event.
- Jobs, in turn are made up of steps.
- A step either runs a shell command or invokes a predefined github **action**. All of the steps in a job are executed on a *runner*.
- The runner is a server(virtual or physical) or a container that has been setup to understand how to interact with GitHub Actions.



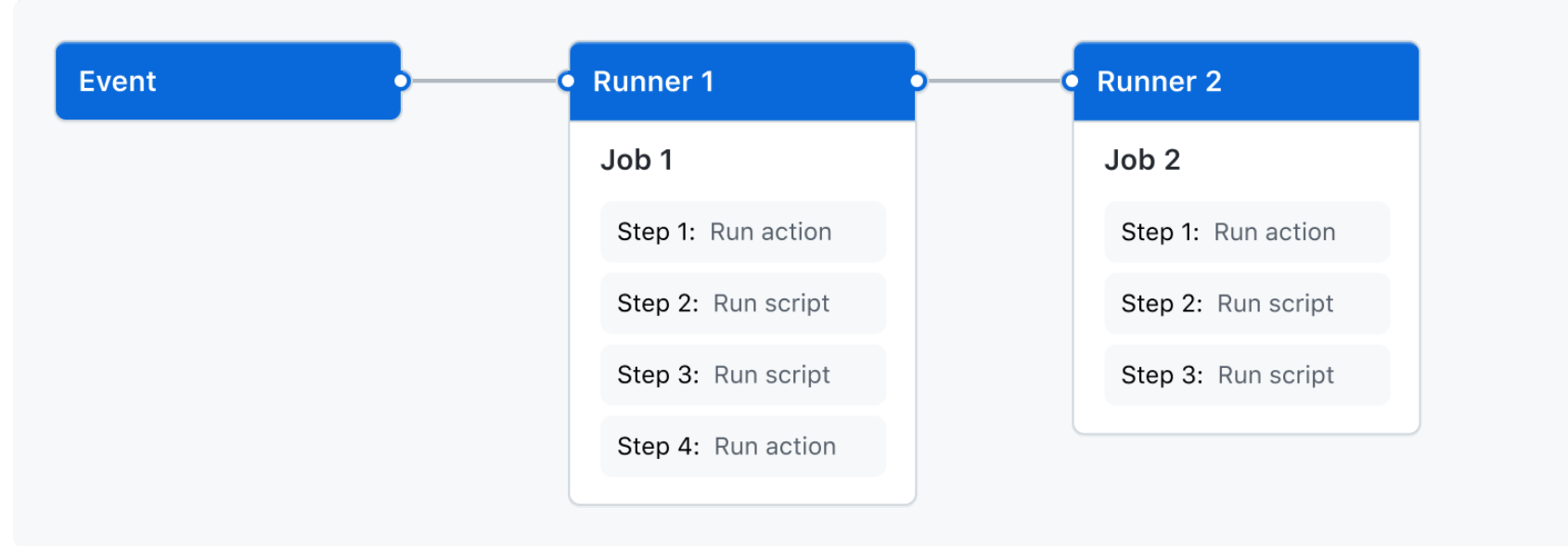
Anatomy of a GitHub Action

A workflow is a unit of automation from its start to finish, including the definition of what triggers the automation (event), what environment or other aspects should be taken into account during the automation, and what should happen due to the trigger. Workflow written in YAML format. Examples of events include forking a repository, pushing code to a remote branch, or opening a pull request.

A job is a section of the workflow and is made up of one or more steps that execute on the same runner/server (ubuntu-latest is the fastest, and cheapest, job runner available.)

A step represents one effect of the automation. Each step consists of either a shell script that's executed, or a reference to an action that's run. When we talk about an action (with a lowercase "a") in this context, we mean a reusable unit of code provided to GitHub, Actions published by the community, or custom actions defined for specific workflows.

Anatomy of a GitHub Action



- Your workflow contains one or more jobs which can run in sequential order or in parallel.
- Each job will run inside its own runner and has one or more steps that either run a script that you define or run an action, which is a reusable extension that can simplify your workflow.
- Steps are executed in order and are dependent on each other. Since each step is executed on the same runner, you can share data from one step to another. For example, you can have a step that builds your application followed by a step that tests the application that was built.
- An **action** performs a complex but frequently repeated task. An action can **pull your Git repository from GitHub**, set up the correct toolchain for your build environment, or set up the authentication to your cloud provider.

Events (using on:)

- Single event: `on: push`
- The workflow can respond to a list (multiple events): `on: [push, pull_request]`
- The workflow can respond to event types with qualifiers, such as branches, tags, or file paths:

```
on:
push:
  branches:
    - main
    - 'rel/v*'
  tags:
    - v1.*
    - beta
  paths:
    - '**.ts'
```

- The workflow can execute on a specific schedule or interval ():
- `on:using standard cron syntax`
`scheduled:`
- `cron: '30 5,15 * * *'`
- The workflow can respond to specific manual events: `on: [workflow-dispatch, repository-dispatch]`
- The workflow can be called from other workflows: `on: workflow_call`
- The workflow can respond to common activities on GitHub items, such as adding a comment to a GitHub issue: `on: issue_comment`

Steps

- Three basic steps in this workflow.
- These steps
 - check out a set of code,
 - set up a go environment based on a particular version, and
 - run the go process on a source file.
- In the YAML syntax, the **- character** indicates where a step starts.
 - The **uses** clause indicates that this step invokes a predefined action.
 - The **with** clause is used to specify arguments/parameters to pass to the action.
 - And the **run** clause indicates a command to be run in the shell.
- Runners are the physical or virtual computers or containers where the code for a workflow is executed. They can be systems provided and hosted by GitHub or they can be instances you set up, host, and control. In a workflow file, runners are defined for jobs simply via the **runs-on** clause.

```
steps:  
- uses: actions/checkout@v3  
- name: setup Go version  
  uses: actions/setup-go@v2  
  with:  
    go-version: '1.14.0'  
- run: go run helloworld.go
```

```
runs-on: ubuntu-latest
```

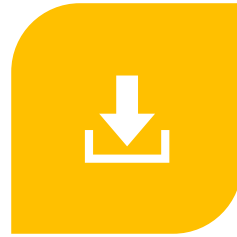
Demo



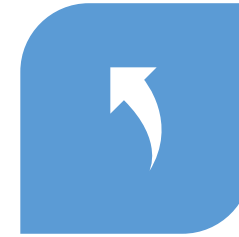
CREATE A
NEW
REPOSITORY



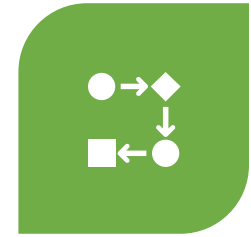
SETUP A
WORKFLOW
(.GITHUB/WORKFLOWS)



ADD A .YML
FILE TO THE
WORKFLOW.
ADD
CONTENT



TRIGGER THE
EVENT



CHECK THE
PROGRESS OF
THE
WORKFLOW
AND JOB
USING LOGS

YAML file content

```
name: Java CI

on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Java
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "17"

      - name: Compile
        run: javac Calculator.java CalculatorTest.java

      - name: Run tests
        run: java CalculatorTest
```

- **on:** Runs the workflow on every **push and pull request**
- **jobs:** Defines the work GitHub should perform
- **runs-on:** Creates a fresh **Ubuntu runner**
- **uses:** Uses existing Actions to **checkout our code and set up Java**
- **run:** Executes our normal terminal commands
- If compilation or the test fails → **the workflow fails** ❌

Demo Flow

- First we write and test code normally.
- Now we put those same checks into a GitHub Actions workflow.
- The workflow runs on a fresh machine, not on our laptop.
- Notice that the pull request gets a red check when the test fails.
- A red check is only information until we enforce it.
- Branch protection turns that failed check into a merge blocker.
- After the fix is pushed, CI reruns and the PR becomes mergeable.

YAML file : another example

```
name: Post welcome comment

on:
  pull_request:
    types: [opened]

permissions:
  pull-requests: write

jobs:
  build:
    name: Post welcome comment
    runs-on: ubuntu-latest
    steps:
      - run: gh pr comment $PR_URL --body "Welcome to the repository!"
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
          PR_URL: ${ github.event.pull_request.html_url }
```

- **permissions** assigns the workflow permissions to operate on the repository
- **pull-requests: write** gives the workflow permission to write to pull requests. This is needed to create the welcome comment.
- **run: gh pr comment \$PR_URL --body "Welcome to the repository!"**
 - This command uses the gh CLI to post a comment on a pull request.
 - gh pr comment is the command used to add a comment to a pull request.
 - \$PR_URL is a variable that holds the URL of the pull request. This URL is used to specify on which pull request the comment should be posted.
 - --body "Welcome to the repository!" specifies the content of the comment.

YAML file: another example

build:

name: Post welcome comment

runs-on: ubuntu-latest

steps:

```
    - run: gh pr comment $PR_URL
      --body "Welcome to the repository!"
```

env:

```
      GITHUB_TOKEN: ${
secrets.GITHUB_TOKEN }
```

```
      PR_URL: ${
github.event.pull_request.html_url
}
```

Environment Variables-

GITHUB_TOKEN:GITHUB_TOKEN is used to authenticate with GitHub to carry out actions that require GitHub permissions, such as commenting on a pull request.

- `${{ secrets.GITHUB_TOKEN }}` retrieves the token from the repository's encrypted secrets. This token provides the necessary permissions to the GitHub Actions runner to interact with the repository on behalf of the user.
- `PR_URL:PR_URL` is set to the HTML URL of the pull request that triggered the workflow.
- `${{ github.event.pull_request.html_url }}` extracts the URL directly from the event data that triggered the workflow. This ensures that the comment is posted to the correct pull request.



Setting secrets

- Secrets allow you to store sensitive information in your organization, repository, or repository environments.
- Create secrets under Settings → Security → Secrets → Click New repository secret → In the Name field, type a name for your secret → In the Secret field, enter the value for your secret → Click Add secret.
- Example: Add an API Key as a Secret

The Result!

Projects Wiki Security Insights Settings

Emoji change to README.md for testing workflow #2

Edit <> Code

Open bharatr21 wants to merge 1 commit into main from test-workflow

Conversation 1 Commits 1 Checks 1 Files changed 1 +1 -1

bharatr21 commented 19 minutes ago Owner

What does this PR do?

(Provide a description of what this PR does.)

Test Plan

(Write your test plan here. If you changed any code, please provide us with clear instructions on how you verified your changes work.)

Related PRs and Issues

(If this PR is related to any other PR or resolves any issue or related to any issue link all related PR and issues here.)

Have you read the Contributing Guidelines on issues?

(Write your answer here.)

Emoji change to README.md for testing workflow Verified ✓ 62e01fb

github-actions bot commented 19 minutes ago Contributor

Welcome to the repository!

Add more commits by pushing to the test-workflow branch on bharatr21/skills-hello-github-actions.

All checks have passed 1 successful check Show all checks

1 participant

Lock conversation

Reviewers

No reviews

Still in progress? Convert to draft

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications Customize

Unsubscribe

You're receiving notifications because you're watching this repository.

GitHub Action for Continuous Integration (CI)

A workflow for linting Markdown files, generating a report in JSON format, and then uploading this report as an artifact.

```
build:
```

```
  runs-on: ubuntu-latest
```

```
  steps:
```

- uses: actions/checkout@v4
- name: Run markdown lint

```
    run: |
```

```
      npm install remark-cli remark-preset-lint-consistent
```

```
      npx remark . --use remark-preset-lint-consistent --frail
```

Generating and uploading test reports

When the work product of one job is needed in another (sequential jobs), we can use the built-in [artifact storage](#) to save artifacts created from one job to be used in another job within the same workflow.

GitHub Action for Continuous Integration (CI)

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Run markdown lint

run: |

npm install remark-cli remark-preset-lint-consistent vfile-reporter-json

npx remark . --use remark-preset-lint-consistent --report vfile-reporter-json 2> remark-lint-report.json

- uses: actions/upload-artifact@v4

with:

name: remark-lint-report

path: remark-lint-report.json

Failing GitHub Action with logs

The screenshot displays the GitHub Actions interface for a workflow named 'Create ci.yml #1'. The workflow is in a failed state, indicated by a red 'X' icon. The left sidebar shows the workflow's summary and a list of jobs, with the 'build' job selected. The main panel shows the logs for the 'build' job, which failed 27 minutes ago in 11s. The logs are organized into steps, with the 'Run markdown lint' step expanded to show its output. The output shows that the 'remark-cli remark-preset-lint-consistent' command was run, and it found no issues in most files. However, it found a warning in 'resume.md' at line 3, column 1, where an 'emphasis marker style' was used incorrectly. The warning message is: 'warning Unexpected emphasis marker `\*`, expected `\_' emphasis-marker remark-lint [cause]: 3:1-3:79 info Emphasis marker style `*_` first defined for `consistent` here emphasis-marker remark-lint'. The logs also show a final error message: 'Error: Process completed with exit code 1.'

<> Code Issues Pull requests 1 Actions Projects Wiki Security Insights Settings

< CI

❌ Create ci.yml #1

Summary

Jobs

❌ build

Run details

Usage

Workflow file

build

failed 27 minutes ago in 11s

Search logs

> Set up job 1s

> Run actions/checkout@v4 1s

> Run a one-line script 0s

> Run a multi-line script 0s

❌ Run markdown lint 8s

```
1 ▶ Run npm install remark-cli remark-preset-lint-consistent
5
6 added 198 packages in 7s
7
8 102 packages are looking for funding
9   run `npm fund` for details
10 .github/steps/0-welcome.md: no issues found
11 .github/steps/1-add-a-test-workflow.md: no issues found
12 .github/steps/2-fix-the-test.md: no issues found
13 .github/steps/3-upload-test-reports.md: no issues found
14 .github/steps/4-add-branch-protections.md: no issues found
15 .github/steps/5-merge-your-pull-request.md: no issues found
16 .github/steps/X-finish.md: no issues found
17 README.md: no issues found
18 resume.md
19 9:18-9:23 warning Unexpected emphasis marker `*`, expected `_' emphasis-marker remark-lint
20 [cause]:
21   3:1-3:79 info Emphasis marker style `*_` first defined for `consistent` here emphasis-marker remark-lint
22
23 ⚠️ 1 warning
24 Error: Process completed with exit code 1.
```

> Post Run actions/checkout@v4 0s

> Complete job 1s

Passing GitHub Action

The screenshot shows the GitHub Actions interface for a workflow named 'Create ci.yml #4'. The workflow is in a 'build' job, which has succeeded 1 minute ago in 13s. The job steps are listed on the right, and the 'Run markdown lint' step is highlighted with a green box. The left sidebar shows the workflow file and other details.

Navigation: <> Code Issues Pull requests 1 Actions Projects Wiki Security Insights Settings

Workflow: Create ci.yml #4 [Re-run all jobs] [More options]

Summary

Jobs

- build

Run details

- Usage
- Workflow file

build
succeeded 1 minute ago in 13s

Search logs

> Set up job	0s
> Run actions/checkout@v4	1s
> Run a one-line script	0s
> Run a multi-line script	0s
> Run markdown lint	9s
> Run actions/upload-artifact@v4	0s
> Post Run actions/checkout@v4	0s
> Complete job	0s

This workflow is ideal for projects that require consistent style and formatting in their Markdown documentation. It ensures that all Markdown files adhere to the specified linting rules, and the lint results are made available for review through the uploaded artifact, which can improve code review processes and maintain code quality.

Summary

•GitHub Actions

- Automates software development workflows directly within GitHub. Triggered by events like push, pull requests, and scheduled times.
- Can be configured to run on various types of events with precise conditions (e.g., branches, tags).

•Components of a GitHub Actions Workflow

- Workflows:** Define automated processes from start to finish in a YAML format.
- Jobs:** Collections of steps within a workflow.
- Steps:** Individual tasks within a job, executed sequentially.
- Actions:** Reusable units of code that perform specific functions in a step.

•Secrets in GitHub Actions

- Used to store sensitive information securely. Configurable at repository or organization levels.
- Critical for maintaining security, especially with API keys and access tokens.

•Practical Implementation: Benefits of GitHub Actions

- Efficiency:** Automates repetitive and complex tasks, reducing manual effort and increasing productivity.
- Reliability:** Ensures consistent execution of deployment and testing workflows, minimizing human errors.
- Scalability:** Easily integrates with existing tools and services, supporting both small projects and large-scale operations.
- Customization:** Highly customizable to meet specific project needs, from simple notifications to full CI/CD pipelines.

Related Project 1 Requirements

- Create 7 PRs ; Review 7 Teammates PRs
- Establish a CI/CD pipeline using GitHub Actions to include automated testing, building, and deployment.

Team-Mentor
Introductions;
OpenAI signups

A solid yellow triangle is located in the bottom right corner of the slide, pointing towards the top right.